
MANUAL TÉCNICO

Frontend

Sistema de Control de Viáticos (SCV)

Gestión de viajes corporativos y rendición de gastos

Autor: Mateo Alejandro Merino Vidal

Empresa: MAXAM FANEXA

Componente: Cliente web (React, Vite y Tailwind CSS)

Versión: 1.0

Fecha: 2 de octubre de 2026

Índice

1. Introducción	3
1.1. Propósito del documento	3
1.2. Descripción general	3
2. Stack tecnológico	3
3. Arquitectura	4
3.1. Capas	4
3.2. Atomic Design	5
3.3. Estructura de carpetas	5
4. Instalación y configuración	5
4.1. Requisitos	6
4.2. Comandos	6
4.3. Variables de entorno	6
4.4. Imagen de ingreso	6
5. Rutas y control de acceso	6
5.1. Guardas	6
5.2. Rutas por perfil	7
6. Gestión de sesión	8
6.1. Token y renovación	8
6.2. Sesión expirada	8
6.3. Cambio de contraseña obligatorio	9
7. Gestión de estado	9
8. Capa de servicios	9
9. Hooks de negocio	10
10. Componentes	11
10.1. Componentes transversales	11
10.2. Componentes por dominio	11
11. Sistema de diseño	12
11.1. Paleta de colores	12
11.2. Colores de estado	12
11.3. Ventanas emergentes	13
12. Funcionalidades destacadas	13
12.1. Planilla de rendición	13
12.2. Control de gasto diario	13
12.3. Digitalización de comprobantes	14
12.4. Revisión y observaciones	14
12.5. Plazos y reemplazos	14
12.6. Administración	14

12.7.Casquito	14
13.Documentación de componentes	14
14.Compilación y despliegue	15
15.Convenciones	15

1. Introducción

1.1 Propósito del documento

Este manual describe la construcción interna del cliente web del Sistema de Control de Viáticos (SCV) de MAXAM FANEXA. Está dirigido a desarrolladores que necesiten instalar, mantener o extender la interfaz, y documenta su arquitectura, organización de carpetas, control de acceso, manejo de sesión, componentes, sistema de diseño y despliegue.

1.2 Descripción general

El cliente es una aplicación de una sola página que atiende a seis perfiles: empleado, supervisor, aprobador, tesorero, revisor y administrador. Cada perfil tiene su propio menú y sus propias pantallas, y todas consumen la API REST del backend. La interfaz es adaptable a escritorio y celular.

Perfil	Funciones principales en la interfaz
Empleado	Crea viajes, registra gastos y facturas, finaliza la rendición, pide reemplazos y ampliaciones de plazo, descarga su planilla
Supervisor	Revisión previa de viajes y revisión de rendiciones de su alcance
Aprobador	Aprueba viajes y revisa rendiciones con alcohol
Tesorero	Asigna y aprueba los fondos de cada viaje
Revisor	Revisión final, solicitudes de plazo y de reemplazo
Administrador	Resumen general, usuarios, cargos, secciones, historial de accesos y recordatorios

2. Stack tecnológico

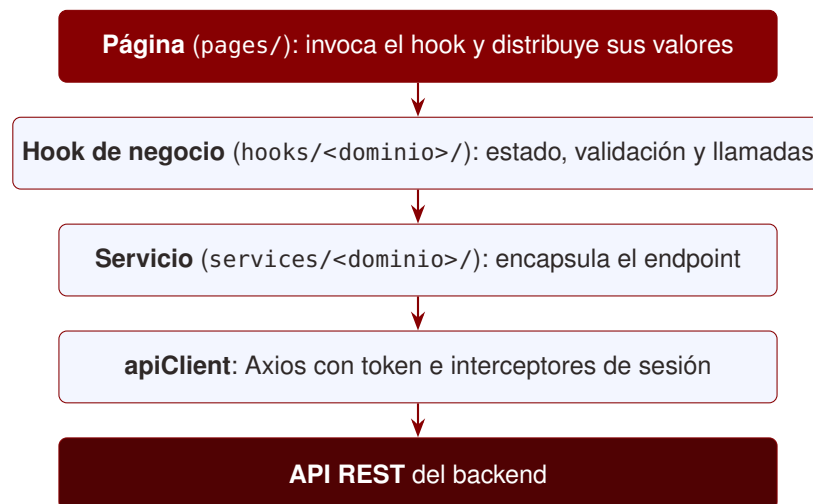
Tecnología	Propósito
React 19	Interfaz mediante componentes funcionales y hooks
Vite	Servidor de desarrollo y compilación
Tailwind CSS 4	Utilidades de maquetado
React Router DOM 7	Enrutamiento y protección por rol
Framer Motion	Animaciones de páginas y ventanas emergentes
Axios	Cliente HTTP con interceptores de sesión
Recharts	Gráficos del panel administrativo
ExcelJS y FileSaver	Planilla de rendición en Excel
Lucide React	Iconografía
jwt-decode	Lectura del rol y del motivo de cambio de contraseña desde el token
Storybook	Documentación interactiva de components/ui

Tecnología	Propósito
ESLint	Análisis estático del código

3. Arquitectura

3.1 Capas

El proyecto combina una organización por dominios con una jerarquía de componentes basada en Atomic Design. La lógica de negocio vive en los hooks; los componentes solo presentan.



Capa	Recibe	Entrega	Restricción
Servicio	Parámetros primitivos	Datos o {error}	No mantiene estado
Hook	Identificadores de ruta	Estado y manejadores	No renderiza JSX
Página	Parámetros de ruta	Árbol de componentes	No llama servicios
Organismo	Props	Fragmento de interfaz	No conoce hooks de negocio

Ningún componente de presentación importa directamente un servicio. Un servicio típico normaliza la respuesta para que el hook solo pregunte por error:

```
export const getTripHistory = async (page, limit, filter) => {
  try {
    const response = await apiClient.get('/trip/history', {params: {pagina: page, limite: limit, filtro: filter}});
    return response.data;
  }
  catch (error) {
    return {error: error.response?.data?.error || 'Error al obtener el historial'};
  }
};
```

3.2 Atomic Design

Nivel	Definición	Ejemplos
Átomo	Elemento indivisible sin lógica de negocio	TripRoute, StatusBadge, ExpenseObsButton
Molécula	Composición que resuelve una unidad funcional	ExpenseBudgetBar, DateRangeFilter, ReturnReviewButton
Organismo	Bloque autónomo y complejo	ExpenseInvoicedTable, ReviewFilters, UserFormModal

3.3 Estructura de carpetas

```
src/
|-- assets/           Imágenes propias (foto-login.jpg)
|-- components/ui/    Componentes genéricos transversales (con Storybook)
|-- features/         Dominios: admin, approval, expense, trip, user
|   '-- <dominio>/
|       |-- atoms/    molecules/  organisms/
|       |-- hooks/     Lógica presentacional
|       '-- constants/ Configuración visual del dominio
|-- hooks/            Hooks de negocio: admin, approval, expense, trip, user, shared
|-- pages/            Páginas asociadas a rutas, y pages/hooks
|-- services/         Acceso a la API por dominio y shared/apiClient.js
|-- layouts/          Navbar, Footer y menús por rol
|-- utils/            Funciones puras (fechas, código de viaje, compresión de imágenes)
|-- constants/        Paleta, rutas, inicio por rol, nombres y colores de estado
|-- App.jsx           Rutas y guardas de acceso
'-- main.jsx          Punto de entrada
```

Todas las páginas, salvo la de ingreso, se cargan con `React.lazy`: cada ruta genera su propio fragmento de JavaScript y las librerías pesadas (ExcelJS, Recharts) no forman parte del paquete inicial.

4. Instalación y configuración

4.1 Requisitos

- Node.js 18 o superior y npm.
- El backend en ejecución y accesible desde el navegador.

4.2 Comandos

```
npm install
npm run dev           # servidor de desarrollo (http://localhost:5173)
npm run build         # compilación para producción (carpeta dist/)
npm run preview       # previsualización de la compilación
npm run lint          # análisis estático
npm run storybook     # documentación de componentes (http://localhost:6006)
```

4.3 Variables de entorno

La única variable es VITE_API_URL, la dirección del backend. Se define en el archivo .env en desarrollo y en las variables del proyecto en Vercel; un cambio requiere volver a compilar.

Etapa	Valor de VITE_API_URL
Local	http://localhost:5000
Piloto	Dirección del servicio en Render
VPS	Dominio de la API, por ejemplo https://api.tuempresa.com

4.4 Imagen de ingreso

La imagen de la pantalla de ingreso es el archivo src/assets/foto-login.jpg, que se incluye en la compilación y no depende de internet. Para cambiarla basta con reemplazar el archivo manteniendo el nombre exacto (Linux y Vercel distinguen mayúsculas y extensión). Se carga con import.meta.glob, de modo que si el archivo aún no está en el repositorio la compilación no falla y se usa la imagen anterior.

5. Rutas y control de acceso

5.1 Guardas

Componente	Comportamiento
ProtectedRoute	Exige sesión y que el rol del token esté entre allowedRoles; si no, lleva al inicio del rol del usuario
LoginRoute	Con sesión activa, la pantalla de ingreso redirige al inicio del rol
TreasurerRoute	Permite las pantallas de tesorería solo a quien tiene el cargo de tesorero

Componente	Comportamiento
PasswordChangeGate	Muestra en cualquier pantalla la ventana de cambio de contraseña obligatorio

Cada rol entra a la primera opción de su menú (`constants/roleHome.js`), y las rutas con parámetros se construyen con funciones de `constants/routes.js` (`tripPath`, `supervisorTripReviewPath`, entre otras) en lugar de textos sueltos.

5.2 Rutas por perfil

Ruta	Página
Empleado (rol 3)	
<code>/dashboard/empleado</code>	Panel: borradores, viajes en curso, reemplazos y recientes
<code>/dashboard/empleado/crear-viaje</code>	Creación de viaje
<code>/dashboard/empleado/viaje/:id</code>	Detalle del viaje según su estado
<code>/dashboard/empleado/viaje/:id/editar</code>	Edición de borrador o viaje rechazado
<code>/dashboard/empleado/viaje/:id/registrar-gasto</code>	Registro de gastos sin factura
<code>/dashboard/empleado/viaje/:id/subir-factura</code>	Carga y lectura de facturas
<code>/dashboard/empleado/gasto/:id</code>	Detalle de un gasto
<code>/dashboard/empleado/historial</code>	Historial de viajes
Supervisor (rol 2)	
<code>/dashboard/supervisor</code>	Rendiciones pendientes
<code>/dashboard/supervisor/historial</code>	Historial de rendiciones revisadas
<code>/dashboard/supervisor/revision/:id</code>	Revisión de una rendición
<code>/dashboard/supervisor/viajes-pendientes</code>	Viajes en revisión previa
<code>/dashboard/supervisor/viaje-previo/:id</code>	Revisión previa de un viaje
Aprobador (rol 5)	
<code>/dashboard/aprobador/revisiones</code>	Viajes por aprobar, aprobados y rechazados
<code>/dashboard/aprobador/revision-alcohol</code>	Rendiciones con alcohol
<code>/dashboard/aprobador/viaje-previo/:id</code>	Aprobación de un viaje
Revisor (rol 4)	
<code>/dashboard/revisor</code>	Rendiciones para revisión final
<code>/dashboard/revisor/revision/:id</code>	Revisión final
<code>/dashboard/revisor/solicitudes-plazo</code>	Solicitudes de ampliación de plazo
<code>/dashboard/revisor/reemplazos</code>	Solicitudes de reemplazo
Tesorero (por cargo)	

Ruta	Página
/dashboard/tesorero/revisiones	Viajes por asignar fondos, aprobados y rechazados
/dashboard/tesorero/viaje/:id	Asignación y aprobación de fondos
Administrador (rol 1)	
/dashboard/administrador	Resumen general con indicadores y gráficos
/dashboard/administrador/usuarios	Gestión de usuarios
/dashboard/administrador/cargos	Gestión de cargos
/dashboard/administrador/secciones	Gestión de secciones
/dashboard/administrador/historial-accesos	Ingresos, salidas y cambios de contraseña
/dashboard/administrador/recordatorios	Días y horas del resumen de pendientes

Todos los perfiles tienen además sus rutas /perfil y /configuracion.

6. Gestión de sesión

6.1 Token y renovación

El token de acceso vive solo en memoria (`services/shared/tokenStore.js`), nunca en `localStorage`. Al abrir la aplicación sin token, `App.jsx` intenta un refresco silencioso con la cookie `httpOnly` antes de mostrar las rutas protegidas.

`apiClient` agrega el token a cada petición y, ante una respuesta Token expirado, pide uno nuevo a `/auth/refresh` y repite la petición. Si varias peticiones vencen a la vez, solo una renueva y las demás esperan en una cola:

```
if (isTokenExpired && !originalRequest._retried) {
  if (isRefreshing) {
    return new Promise((resolve, reject) => {
      waitingQueue.push({resolve, reject});
    }).then((token) => {
      originalRequest.headers.Authorization = `Bearer ${token}`;
      return apiClient(originalRequest);
    });
  }
  originalRequest._retried = true;
  isRefreshing = true;
}
```

6.2 Sesión expirada

Si la renovación falla (cookie vencida, sesión invalidada por cambio de rol, suspensión o cambio de contraseña en otro equipo), `apiClient` borra el token y emite el evento `session-expired`. `useMenu` lo escucha y muestra la ventana “Sesión Expirada”, presente en todas las pantallas salvo la de ingreso; su botón cierra la sesión y vuelve al ingreso. `useMenu` además compara el rol cada 30 segundos.

6.3 Cambio de contraseña obligatorio

Cuando el servidor responde 403 con CAMBIO_CONTRASENIA_REQUERIDO, apiClient emite password-change-required y PasswordChangeGate muestra la ventana según el motivo del token:

Motivo	Texto de la ventana	Campos
TEMPORAL	Contraseña temporal: crea una propia	Nueva y confirmación
RECUPERACION	Ingresaste con un código de verificación	Nueva y confirmación
VENCIDA	Actualiza tu contraseña (90 días)	Actual, nueva y confirmación; ofrece cerrar sesión

Mientras está activa, los paneles ignoran los errores causados por el bloqueo para no mostrar avisos falsos.

7. Gestión de estado

No se usa una biblioteca de estado global: el estado es local a cada pantalla y vive en su hook de negocio.

- **Servidor:** se obtiene al montar y se refresca tras cada escritura.
- **Formularios:** el hook guarda los valores y los errores por campo.
- **Sondeo:** cada 30 segundos se actualizan las bandejas de revisión, las solicitudes de plazo y reemplazo, el historial de viajes del empleado y los resúmenes. Mientras el empleado tiene una solicitud esperando respuesta, también se consulta cada 30 segundos. Solo la carga inicial muestra el indicador esquelético; si una actualización falla se conservan los datos y se muestra un aviso.
- **Concurrencia:** si otra persona ya procesó un viaje o una solicitud, el servidor responde 409 y la interfaz lo explica y actualiza la lista.

8. Capa de servicios

Servicio	Endpoints que encapsula
user/authService	Ingreso, cierre de sesión, códigos de recuperación
user/userService	Perfil, foto, contraseña, usuarios, empleados
trip/tripService	Panel, historial, detalle, creación, edición, envío, finalización y planilla PDF
expense/expenseService	Gastos, detalle y recibos
expense/invoiceService	Extracción, registro y edición de facturas

Servicio	Endpoints que encapsula
approval/reviewService	Revisión del supervisor (viajes y rendiciones)
approval/approverService	Aprobación de viajes
approval/ approverAlcoholReviewService	Revisión por alcohol
approval/treasurerService	Asignación y aprobación de fondos
approval/reviewerService	Revisión final
approval/ deadlineAuthorizationService	Solicitudes de ampliación de plazo
approval/substitutionService	Solicitudes de reemplazo
admin/adminService	Resumen general, cargos, secciones, historial de accesos y recordatorios

9. Hooks de negocio

Dominio	Hooks
trip	useEmployeeDashboard, useCreateTrip, useEditTrip, useTripDetail, useTripHistory, useStatementPdfDownload
expense	useRegisterExpense, useEditExpense, useUploadInvoice, useEditInvoice, useInvoiceForm, useInvoiceDetailPanel
approval	Uno por bandeja y por detalle de cada revisor (useSupervisorPendingTrips, useApproverReviews, useTreasurerReviews, useReviewerReviews, ...), más useDeadlineAuthorization, useSubstitutionRequest y sus pares del revisor
admin	useAdminDashboard, useUserManagement, usePositionManagement, useSectionManagement, useAuditLog, useReminderSettings
user	useLogin, useProfile, useCurrentUser, useIsTreasurer, usePasswordExpiredCheck
shared	useMenu, useMenuNavigation, useClientPagination, useSimpleSelector, useUserRole

Ejemplo de hook con validación previa y ventana de aviso, tomado de la finalización de un viaje:

```

const handleRequestSubmitReview = () => {
  if (expenses.length === 0) {
    showError('El viaje debe tener por lo menos un gasto asociado para ser finalizado');
    return;
  }
  const {missingDays, missingHotel} = missingJustifications();
  if (missingDays.length > 0 || missingHotel) {
    showError('Debes justificar cada día que excede la cuota diaria, y el exceso en hoteles si corresponde');
    return;
  }
  setShowSubmitReviewModal(true);
};

```

10. Componentes

10.1 Componentes transversales

Componente	Uso
ConfirmDialog	Ventana emergente de confirmación o aviso; se dibuja en un portal sobre toda la pantalla
Button, ModalActions	Botones primario y secundario
ModalIconHeader	Ícono en círculo blanco de las ventanas
InputField, PasswordField, FormField, LimitedTextarea	Campos de formulario
EmployeeDropdown, SectionDropdown, InlineDropdown	Selectores con búsqueda
ListCount, LoadMoreButton	“Mostrando X de Y” y paginación
SkeletonCard, SkeletonList	Indicadores de carga
EmptyState, InlineAlert, SuccessModal	Estados vacíos y avisos
PageHeader, AppHeader, AppLogo	Encabezados
CommentCard	Observación de un revisor

10.2 Componentes por dominio

Dominio	Organismos principales
trip	TripActiveExpenseView, TripDraftView, TripFinalApprovedView, TripBudgetSummary, TripHistoryFilter, SubstitutionRequestModal
expense	InvoiceForm, InvoiceDropZone, InvoiceDetailPanel, ExpenseFormCard, ExpenseWithholdingsCard, ReceiptSentModal
approval	ReviewFilters, PendingTripItem, ExpenseInvoicedTable, ExpenseUninvoicedTable, ExpenseInternationalTable, SelectableObservationsList, TreasurerAmountsCard
admin	UserFormModal, PositionFormModal, SectionFormModal, TripPhaseSection
user	PasswordChangeGate, PasswordExpiredModal, SessionExpiredModal, VerificationCodeModal, TermsModal

11. Sistema de diseño

11.1 Paleta de colores

La paleta se centraliza en `constants/index.js`; ningún componente define colores sueltos.

Color	Token	Valor	Uso
	primary	#870002	Acciones principales y ventanas emergentes
	secondary	#D20F12	Botón de confirmación, errores
	title	#500203	Títulos
	text	#2E2827	Texto
	labels	#475569	Etiquetas y texto secundario
	background	#FFFFFF	Fondo general
	backgroundSecondary	#000000	Íconos sobre círculo blanco
	backgroundHeader	#F3F6FF	Tarjetas y encabezados de tabla
	dataFields	#DEE2F0	Bordes y separadores
	fields	#C2C6D4	Controles deshabilitados
	error	#FDE9E9	Fondo de mensajes de error

Tipografías: **Inter** para el contenido y **Nunito** para los botones.

11.2 Colores de estado

Cada estado de viaje tiene un único nombre (`constants/tripStatusLabel.js`) y un único par de colores (`constants/tripStatusColors.js`) en todas las pantallas. Los avisos de confirmación o rechazo usan el color del estado en que queda el viaje.

Estado	Etiqueta	Significado
BORRADOR	Borrador	Aún no enviado
EN_REVISION_VIAJE	Revisión Previa	Con el supervisor
APROBADO_VIAJE	Esperando Aprobador	Con el aprobador
EN_REVISION_TESORERO	Esperando Fondos	Con tesorería
EN_CURSO	En Curso	Registro de gastos
EN_REVISION	En Revisión	Rendición con el supervisor
EN_REVISION_APROBADOR	Revisión por Alcohol	Con el aprobador
APROBADO_SUPERVISOR	Apr. Preliminar	Con el revisor final
APROBADO_FINAL	Aprobado	Cerrado
RECHAZADO	Rechazado	Debe corregirse

En las pantallas del empleado, “En Curso” usa el rojo institucional porque marca su viaje activo.

11.3 Ventanas emergentes

Todas las confirmaciones y avisos importantes usan `ConfirmDialog`: tarjeta en rojo institucional, ícono negro en un círculo blanco y botones primario y secundario. Se dibuja con `createPortal` sobre el body, lo que evita parpadeos cuando se abre desde una tarjeta animada.

```
<ConfirmDialog isOpen={!!result} compact
  icon={result?.success ? CheckCircle : AlertCircle} iconColor={COLORS.primary}
  title={result?.success ? 'Listo' : 'No se pudo completar'}
  message={result?.message} confirmText="Entendido" hideCancel onConfirm={closeResult} />
```

Se muestran en ventana, entre otros: los errores al finalizar un viaje, la devolución de una revisión, el resultado del envío de recibos, las respuestas a solicitudes y los avisos de solicitudes cerradas automáticamente.

12. Funcionalidades destacadas

12.1 Planilla de rendición

`useTripExcelExport` construye la planilla oficial en Excel: logotipo, simbología, distinción entre gastos nacionales e internacionales, cuenta Oracle por categoría, tramos de cambio, totales por moneda, justificaciones de exceso y cinco bloques de firma. El empleado descarga en cambio la planilla en PDF, generada por el backend, cuando el viaje está aprobado. Ambos formatos usan las retenciones persistidas por el backend.

12.2 Control de gasto diario

El detalle del viaje muestra el desglose por día contra el viático del cargo. Cuando un día o el total de hoteles excede la cuota, `useTripDetail` exige una justificación por cada exceso antes de permitir el

envío a revisión.

12.3 Digitalización de comprobantes

La carga de facturas presenta cada comprobante en tres columnas (imagen, formulario y detalle de productos) para corregir los datos extraídos antes de guardar. Cada factura tiene su propio estado: en extracción, con datos, con error o guardada. La fecha de emisión queda bloqueada si se extrajo con formato válido y editable si vino vacía o mal formada. Las imágenes se comprimen en el navegador antes de enviarse.

12.4 Revisión y observaciones

Cada bandeja de revisión tiene las pestañas Pendientes, Aprobados y Rechazados, con filtros por fecha, empleado y sección, y muestra lo revisado más reciente primero, de 12 en 12. Para rechazar se seleccionan observaciones propias de la ronda actual; las de otros revisores se ven, pero no se pueden editar, borrar ni usar para rechazar. El supervisor y el aprobador pueden devolver una revisión tomada por error.

12.5 Plazos y reemplazos

`useDeadlineAuthorization` determina si venció el plazo de carga (cuatro días de tolerancia más las ampliaciones vigentes) y ofrece pedir una ampliación. El plazo es del viaje, por lo que el reemplazo ve y puede pedir la misma ampliación. `useSubstitutionRequest` gestiona el pedido de reemplazo; el viaje aparece en el panel del reemplazo como “Rendición de [nombre]”. Las solicitudes cerradas por el sistema se muestran como “Cerrada” y no como rechazadas.

12.6 Administración

Resumen general con indicadores por fase y gráficos; gestión de usuarios con buscador en cargo, sección y jefe directo (`SearchableSelector`); sugerencias de nombres existentes al crear cargos y secciones; historial de accesos con filtros, paginación y exportación a Excel; y la pantalla de *Recordatorios*, donde el administrador activa o desactiva el resumen de pendientes, elige los días de la semana y hasta cuatro horas de envío (hora de Bolivia), ve una vista previa de a quién le llegaría y puede enviarlo en el momento. Los cambios se aplican al guardar, sin reiniciar el servidor.

12.7 Casquito

Mascota que saluda una vez por pantalla y sesión con un mensaje según la hora y los pendientes. Se puede desactivar en Ajustes de Cuenta y respeta la preferencia de reducir movimiento.

13. Documentación de componentes

Los componentes de `components/ui` están documentados en Storybook. Cada historia describe el propósito del componente y sus variantes; los que tienen estado interno (selectores, diálogos) incluyen

un contenedor interactivo para probarlos.

```
npm run storybook      # http://localhost:6006
npm run build-storybook # sitio estático de la documentación
```

14. Compilación y despliegue

1. Definir `VITE_API_URL` con la dirección del backend de la etapa.
2. Verificar que `src/assets/foto-login.jpg` esté en el repositorio.
3. En Vercel: proyecto con Vite, comando `npm run build` y carpeta `dist`. Cada cambio de variable requiere volver a desplegar.
4. En un servidor propio: `npm run build` y servir `dist/` con Nginx, redirigiendo todas las rutas a `index.html`.
5. Agregar el dominio del cliente en `CORS_ORIGINS` del backend si no es el de Vercel.

15. Convenciones

- El código se escribe en inglés y los textos visibles en español.
- Los componentes definen sus estilos en un objeto `styles` con clases de Tailwind y toman los colores de `COLORS`.
- En el frontend no se agregan comentarios por función; el nombre y el contenido deben bastar.
- Se prefieren bloques `if/else` explícitos; el operador ternario solo se usa en valores de estilo o del JSX.
- Punto y coma al final de cada sentencia.
- Los commits siguen Conventional Commits: `feat`, `fix`, `style`, `refactor`, `docs`, `test` y `chore`.